

Reviewer Pack — Cohomological Dimension Inverse Proportional to Disjointness...

Entry 71110706deb9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 22:59:19 UTC

Cohomological Dimension Inverse Proportional to Disjointness Communication Complexity

Entry ID: 71110706deb9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 22:59:19 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Cohomology of Simplicial Complexes) **Field B** (complexity object): Communication Complexity of Set Disjointness

Statement:

For a random disjointness instance with n variables, the cohomological dimension of the simplicial complex formed by its communication matrix is $\Theta(\log n / \log \log n)$ times the deterministic communication complexity. This holds for all instances where the matrix is a complete bipartite graph.

Rationale (proposer's reasoning):

Cohomological dimension captures the 'thickness' of topological spaces, which may mirror the information-theoretic constraints in communication protocols. The link arises via simplicial complexes encoding interaction patterns between parties.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5467581dc957dd00

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    B = [[A[i][j] if i < n // 2 and j >= n // 2 else 0 for j in range(n)] for i in range(n)]
    C = [[A[i][j] if i >= n // 2 and j < n // 2 else 0 for j in range(n)] for i in range(n)]

    def matrix_multiplication(A, B):
        result = [[sum(a * b for a, b in zip(row_A, col_B)) for col_B in zip(*B)] for row_A in A]
        return result

    CC = 0
    for i in range(n // 2):
        for j in range(n // 2, n):
            if B[i][j] == 1:
                CC += 1

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for k in range(i + 1, m):
                if abs(A[k][i]) > abs(A[max_row][i]):
                    max_row = k
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for k in range(m):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]

    return A
```

```

def rank(A):
    A = gaussian_elimination(A)
    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def simplicial_complex_rank(n):
    return n - 1

cohomological_dimension = simplicial_complex_rank(rank(B))

conjecture_holds = abs(cohomological_dimension - (math.log(n) / math.log(math.log(n)))) < 0.1
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "cohomological_dimension",
    "metric_value": cohomological_dimension,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ebb4112.py", line 83, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ebb4112.py", line 64, in run_trial
    cohomological_dimension = simplicial_complex_rank(rank(B))
                               ^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ebb4112.py", line 54, in rank
    A = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ebb4112.py", line 45, in gaussian_elim
    A[i][j] /= factor
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division by zero error, preventing data collection. The pre-registered support condition cannot be evaluated without valid results. | next: Debug Gaussian elimination implementation to handle zero factors, then re-run tests with controlled parameter ranges

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113022
2	propose	ollama_remote	qwen3:8b	0	0	67797
3	propose	ollama_remote	qwen3:8b	0	0	111812
4	propose	ollama_remote	qwen3:8b	0	0	116565
5	propose	ollama_remote	qwen3:8b	0	0	111656
6	propose	ollama_remote	qwen3:8b	0	0	88450
7	preregistration	ollama_remote	qwen3:8b	0	0	24187
8	novelty	ollama_remote	qwen3:8b	0	0	20825
9	novelty	ollama_remote	qwen3:8b	0	0	12979
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14603
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10358
12	judge	ollama_remote	qwen3:8b	0	0	18469

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 710723 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in [research/audit/71110706deb9.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/71110706deb9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/71110706deb9.tar.gz` (if generated)